

Vježba: Izrada prijavne forme pomoću **DynamicForm** komponente

Naziv projekta

08_React/06_Practice_DynamicCourseApplicationForm/

Cilj vježbe

U ovoj vježbi učenici neće ručno pisati svaki input u JSX-u. Umjesto toga, koristit će već pripremljenu **DynamicForm** komponentu.

Njihov zadatak je izraditi React aplikaciju za prijavu na online tečaj. Forma se mora generirati pomoću **fields** konfiguracije.

Učenici će vježbati:

- korištenje postojeće reusable komponente
 - prosljeđivanje propsa komponenti
 - izradu **fields** arraya
 - konfiguriranje različitih tipova polja
 - korištenje validacija kroz konfiguraciju
 - obradu podataka kroz **onSubmit**
 - prikaz summary kartice nakon uspješnog slanja forme
 - razumijevanje kako roditeljska komponenta komunicira s dinamičkom formom
-

Scenarij

Zamislite da izrađujete stranicu za prijavu na React tečaj.

Korisnik treba ispuniti prijavnu formu s osnovnim podacima. Forma mora provjeriti jesu li obavezna polja unesena, je li email ispravan, jesu li godine u dopuštenom rasponu i je li motivacijska poruka dovoljno duga.

Forma se ne smije pisati ručno kao niz inputa u `App.jsx`.

Umjesto toga, svi inputi moraju nastati iz konfiguracije koju šaljemo `DynamicForm` komponenti.

Polazna točka

Učenici već imaju gotovu komponentu:

```
src/DynamicForm.jsx
```

Ta komponenta prima dva propsa:

```
<DynamicForm fields={fields} onSubmit={handleSubmit} />
```

`fields` je array objekata koji opisuje polja forme.

`onSubmit` je funkcija koja se poziva samo kada je forma uspješno validirana.

Zadatak

1. Kreirajte novi React projekt

U terminalu pokrenite:

```
npm create vite@latest dynamic-course-application-form -- --template react
```

Zatim uđite u projekt:

```
cd dynamic-course-application-form
```

Instalirajte pakete:

```
npm install
```

Pokrenite aplikaciju:

```
npm run dev
```

2. Dodajte **DynamicForm** komponentu

U folderu **src** napravite novu datoteku:

DynamicForm.jsx

U nju zalijepite gotovu **DynamicForm** komponentu iz lekcije.

Struktura projekta treba izgledati ovako:

```
src/  
  App.jsx  
  App.css  
  DynamicForm.jsx  
  main.jsx
```

3. Pripremite osnovni **App.jsx**

U **src/App.jsx** importajte **useState**, **CSS** i **DynamicForm**.

```
import { useState } from "react";  
import DynamicForm from "../DynamicForm";  
import "../App.css";
```

```
function App() {  
  return (  
    <main className="app">  
      <h1>React Course Application</h1>  
    </main>  
  );  
}
```

```
export default App;
```

4. Dodajte state za spremljenu prijavu

U `App` komponenti dodajte state koji će čuvati podatke uspješno poslane forme:

```
const [applicationData, setApplicationData] = useState(null);
```

Na početku je vrijednost `null` zato što forma još nije poslana.

Kada korisnik uspješno pošalje formu, u ovaj state spremit ćemo podatke koje dobijemo iz `DynamicForm` komponente.

5. Napravite `fields` konfiguraciju

Unutar `App` komponente napravite array `fields`.

Forma mora imati ova polja:

- Full name
- Email
- Age
- Experience level
- Preferred course format
- Motivation
- Newsletter
- Accept terms

Važno: nemojte ručno pisati inpute u JSX-u. Sva polja moraju biti definirana u `fields` arrayu.

6. Dodajte polje `Full name`

Polje treba biti tekstualni input.

Pravila:

Obavezno je.

Mora imati barem 3 znaka.
Maksimalno 50 znakova.

Konfiguracija:

```
{
  name: "fullName",
  label: "Full name",
  type: "text",
  required: true,
  placeholder: "Enter your full name",
  minLength: {
    enabled: true,
    value: 3,
    message: "Full name must have at least 3 characters."
  },
  maxLength: {
    enabled: true,
    value: 50,
    message: "Full name can have at most 50 characters."
  }
}
```

7. Dodajte polje **Email**

Polje treba biti tekstualni input.

Pravila:

Obavezno je.
Mora biti ispravna email adresa.

Konfiguracija:

```
{
  name: "email",
  label: "Email",
  type: "text",
  required: true,
  placeholder: "name@example.com",
```

```
email: {
  enabled: true,
  message: "Please enter a valid email address."
}
}
```

8. Dodajte polje **Age**

Polje treba biti brojčani input.

Pravila:

Obavezno je.

Vrijednost mora biti između 16 i 80.

Konfiguracija:

```
{
  name: "age",
  label: "Age",
  type: "number",
  required: true,
  placeholder: "Enter your age",
  range: {
    enabled: true,
    min: 16,
    max: 80,
    message: "Age must be between 16 and 80."
  }
}
```

9. Dodajte **select** za razinu iskustva

Polje treba biti **select**.

Opcije:

Beginner

Intermediate
Advanced

Pravila:

Obavezno je.

Konfiguracija:

```
{  
  name: "experienceLevel",  
  label: "Experience level",  
  type: "select",  
  required: true,  
  placeholder: "-- choose your level --",  
  options: ["Beginner", "Intermediate", "Advanced"]  
}
```

10. Dodajte radio grupu za format tečaja

Polje treba biti **radio**.

Opcije:

Online
Classroom
Hybrid

Pravila:

Obavezno je.

Konfiguracija:

```
{  
  name: "courseFormat",  
  label: "Preferred course format",  
  type: "radio",  
  required: true,  
}
```

```
options: ["Online", "Classroom", "Hybrid"]
}
```

11. Dodajte **textarea** za motivaciju

Polje treba biti **textarea**.

Pravila:

Obavezno je.

Mora imati barem 30 znakova.

Može imati najviše 300 znakova.

Konfiguracija:

```
{
  name: "motivation",
  label: "Motivation",
  type: "textarea",
  required: true,
  placeholder: "Tell us why you want to join this course...",
  rows: 5,
  minLength: {
    enabled: true,
    value: 30,
    message: "Motivation must have at least 30 characters."
  },
  maxLength: {
    enabled: true,
    value: 300,
    message: "Motivation can have at most 300 characters."
  }
}
```

12. Dodajte checkbox za newsletter

Polje treba biti checkbox, ali nije obavezno.

Konfiguracija:

```
{
  name: "newsletter",
  label: "Newsletter",
  type: "checkbox",
  checkboxLabel: "I want to receive course news and updates"
}
```

13. Dodajte checkbox za prihvatanje uvjeta

Polje treba biti checkbox i mora biti obavezno.

Konfiguracija:

```
{
  name: "acceptTerms",
  label: "Terms and conditions",
  type: "checkbox",
  required: true,
  checkboxLabel: "I accept the terms and conditions"
}
```

14. Napravite **handleSubmit** funkciju

Dodajte funkciju koja prima podatke iz **DynamicForm** komponente i sprema ih u **applicationData**.

```
function handleSubmit(data) {
  setApplicationData(data);
}
```

Ova funkcija će se pozvati samo ako je forma prošla validaciju.

15. Prikažite formu

U JSX-u prikažite `DynamicForm` komponentu.

```
<DynamicForm fields={fields} onSubmit={handleSubmit} />
```

Ako `applicationData` još ne postoji, prikažite formu.

Ako postoji, prikažite summary karticu.

16. Prikažite summary nakon uspješnog slanja

Ako je forma uspješno poslana, aplikacija treba prikazati podatke:

Application Summary

Full name: ...

Email: ...

Age: ...

Experience level: ...

Preferred course format: ...

Motivation: ...

Newsletter: Yes / No

Terms accepted: Yes / No

Dodajte i gumb:

Submit another application

Klik na taj gumb treba vratiti `applicationData` na `null`.

```
<button onClick={() => setApplicationData(null)}>  
  Submit another application  
</button>
```

Očekivano ponašanje aplikacije

Kada korisnik otvori aplikaciju, vidi naslov i formu za prijavu.

Ako klikne **Pošalji** bez unosa podataka, moraju se prikazati validacijske greške.

Ako korisnik počne tipkati u polja, greške se moraju ažurirati tijekom unosa.

Ako korisnik napusti polje, validacija se također pokreće.

Ako su svi podaci ispravni i korisnik klikne **Pošalji**, forma nestaje i prikazuje se summary kartica.

Ako korisnik klikne **Submit another application**, aplikacija ponovno prikazuje praznu formu.

Puno rješenje

src/App.jsx

```
import { useState } from "react";
import DynamicForm from "../DynamicForm";
import "../App.css";

function App() {
  const [applicationData, setApplicationData] = useState(null);

  const fields = [
    {
      name: "fullName",
      label: "Full name",
      type: "text",
      required: true,
      placeholder: "Enter your full name",
      minLength: {
        enabled: true,
        value: 3,
        message: "Full name must have at least 3 characters."
      },
      maxLength: {
        enabled: true,
        value: 50,
        message: "Full name can have at most 50 characters."
      }
    },
  ],
```

```
{
  name: "email",
  label: "Email",
  type: "text",
  required: true,
  placeholder: "name@example.com",
  email: {
    enabled: true,
    message: "Please enter a valid email address."
  }
},
{
  name: "age",
  label: "Age",
  type: "number",
  required: true,
  placeholder: "Enter your age",
  range: {
    enabled: true,
    min: 16,
    max: 80,
    message: "Age must be between 16 and 80."
  }
},
{
  name: "experienceLevel",
  label: "Experience level",
  type: "select",
  required: true,
  placeholder: "-- choose your level --",
  options: ["Beginner", "Intermediate", "Advanced"]
},
{
  name: "courseFormat",
  label: "Preferred course format",
  type: "radio",
  required: true,
  options: ["Online", "Classroom", "Hybrid"]
},
{
  name: "motivation",
  label: "Motivation",
  type: "textarea",
  required: true,
```

```

placeholder: "Tell us why you want to join this course...",
rows: 5,
minLength: {
  enabled: true,
  value: 30,
  message: "Motivation must have at least 30 characters."
},
maxLength: {
  enabled: true,
  value: 300,
  message: "Motivation can have at most 300 characters."
}
},
{
  name: "newsletter",
  label: "Newsletter",
  type: "checkbox",
  checkboxLabel: "I want to receive course news and updates"
},
{
  name: "acceptTerms",
  label: "Terms and conditions",
  type: "checkbox",
  required: true,
  checkboxLabel: "I accept the terms and conditions"
}
];

```

```

function handleSubmit(data) {
  setApplicationData(data);
}

```

```

if (applicationData !== null) {
  return (
    <main className="app">
      <section className="summary-card">
        <h1>Application Summary</h1>

        <p>
          <strong>Full name:</strong> {applicationData.fullName}
        </p>

        <p>
          <strong>Email:</strong> {applicationData.email}

```

```

    </p>

    <p>
      <strong>Age:</strong> {applicationData.age}
    </p>

    <p>
      <strong>Experience level:</strong> {applicationData.experienceLevel}
    </p>

    <p>
      <strong>Preferred course format:</strong>{" "}
      {applicationData.courseFormat}
    </p>

    <p>
      <strong>Motivation:</strong> {applicationData.motivation}
    </p>

    <p>
      <strong>Newsletter:</strong>{" "}
      {applicationData.newsletter ? "Yes" : "No"}
    </p>

    <p>
      <strong>Terms accepted:</strong>{" "}
      {applicationData.acceptTerms ? "Yes" : "No"}
    </p>

    <button onClick={() => setApplicationData(null)}>
      Submit another application
    </button>
  </section>
</main>
);
}

return (
  <main className="app">
    <h1>React Course Application</h1>

    <p className="intro">
      Fill out the form below to apply for the React course.
    </p>

```

```
      <DynamicForm fields={fields} onSubmit={handleSubmit} />
    </main>
  );
}

export default App;
```

src/App.css

```
body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #eef2f7;
}

.app {
  max-width: 680px;
  margin: 50px auto;
  padding: 32px;
  background-color: white;
  border: 1px solid #d8dee9;
  border-radius: 16px;
}

h1 {
  margin-top: 0;
  text-align: center;
}

.intro {
  text-align: center;
  color: #526070;
  margin-bottom: 28px;
}

.form-row {
  margin-bottom: 18px;
}

label {
  display: block;
```

```
    margin-bottom: 8px;
    font-weight: bold;
}
```

```
.required {
    color: #c0392b;
}
```

```
input,
select,
textarea {
    width: 100%;
    box-sizing: border-box;
    padding: 10px;
    border: 1px solid #ccd3df;
    border-radius: 8px;
    font-size: 16px;
    font-family: inherit;
}
```

```
textarea {
    resize: vertical;
}
```

```
.checkbox-group label,
.radio-group label {
    display: flex;
    align-items: center;
    gap: 8px;
    font-weight: normal;
    margin-bottom: 8px;
}
```

```
.checkbox-group input,
.radio-group input {
    width: auto;
}
```

```
.input-validation-error {
    border-color: #c0392b;
    background-color: #fff7f5;
}
```

```
.error {
```



```
margin: 6px 0 0;  
color: #c0392b;  
font-size: 14px;  
}  
  
button {  
padding: 10px 16px;  
border: 1px solid #ccd3df;  
border-radius: 8px;  
background-color: #f7f9fc;  
cursor: pointer;  
font-size: 15px;  
}  
  
button:hover {  
background-color: #e9eef6;  
}  
  
.summary-card p {  
line-height: 1.6;  
}
```

Dodatni izazovi

Izazov 1: Dodajte phone number polje

Dodajte novo polje:

Phone number

Pravila:

Obavezno je.

Smije sadržavati samo brojeve, razmake i znak +.

Koristite **pattern** validaciju:

```
{  
name: "phone",
```

```
label: "Phone number",
type: "text",
required: true,
placeholder: "+385 91 123 4567",
pattern: "[0-9+\\s]+$",
patternMessage: "Phone number can contain only numbers, spaces and +."
}
```

Zatim prikazite phone number u summary kartici.

Izazov 2: Dodajte custom validator za email domenu

Email mora završavati s:

@example.com

Dodajte **validator** funkciju u email field:

```
validator: (value) => {
  if (!value.endsWith("@example.com")) {
    return "Email must use the @example.com domain.";
  }

  return "";
}
```

Ovdje učenici trebaju vidjeti da custom validator omogućuje pravila koja nisu unaprijed ugrađena u **DynamicForm**.

Izazov 3: Dodajte polje za potvrdu emaila

Dodajte novo polje:

Confirm email

Polje mora biti obavezno i mora se podudarati s emailom.

```
{
  name: "confirmEmail",
  label: "Confirm email",
  type: "text",
  required: true,
  placeholder: "Repeat your email address",
  validator: (value, formData) => {
    if (value !== formData.email) {
      return "Email addresses do not match.";
    }

    return "";
  }
}
```